

PROJECT AND TEAM INFORMATION

Project Title

ForceForge: An Interactive 2D Physics Simulation Sandbox

Student / Team Information

<i>Team Name:</i> <i>Team ID:</i>	<i>Team Jkd</i> <i>DSCPP-III-2026-T098</i>
Team member 1 (Team Lead)	<i>Negi, Pranjali</i> <i>Student ID: 2510370602</i> <i>2510370602@gau.ac.in</i> 
Team member 2	<i>Rawat, Dilasha</i> <i>Student ID: 2510012530</i> <i>2510012530@geu.ac.in</i> 
Team member 3	<i>Jakhmola, Rishabh</i> <i>Student ID: 25101210555</i> <i>25101210555@geu.ac.in</i> 
Team member 4	<i>Chhabra, Akshit</i> <i>Student ID: 2510011905</i> <i>2510011905@geu.ac.in</i> 

PROPOSAL DESCRIPTION

Motivation

Physics concepts like gravity, friction, momentum and collisions are often taught using equations and static diagrams. This makes it hard for students to understand how these forces actually behave over time. A student might memorize the formula for a collision. Never see what happens when two objects with different mass and velocity actually hit each other. That gap between math and real motion can make mechanics feel confusing. The Physics Simulation Sandbox is designed to close that gap. It creates a 2D environment where students can spawn particles, circles and boxes change their properties and watch physical behavior happen in time. Users will be able to experiment with gravity, friction, momentum, collisions and projectile motion of just reading about them. The project gives feedback so students can connect physics equations to actual movement. It also lets users tweak values like mass, velocity, friction, restitution and gravity directly. This hands-on experience makes learning more engaging. It helps students build an understanding of Newtonian mechanics by seeing cause and effect.

State of the Art / Current Solution

There are already some tools that help visualize physics for learners. PhET Interactive Simulations from the University of Colorado Boulder offers browser-based simulations on topics like gravity, friction and collisions.. Each simulation focuses on one idea only. They don't let users combine forces or mix behaviors easily. Algodoo is a 2D physics sandbox where people can play with objects and interactions. However it has features beyond basic mechanics—like scripting and advanced simulations—that could distract students who want to focus on core ideas. On the hand JavaScript physics engines such as Matter.js and Box2D-based libraries are used to build custom simulations. These work well. Are meant for developers, not education. They lack an interface for teaching. Our project aims to create something open and focused on teaching mechanics. It will also show programming concepts like object-oriented design, data structures, collision algorithms and physics simulation techniques.

Project Goals and Milestones

The main goal is to build a 2D physics sandbox that lets students see physics in action. Students should be able to grasp ideas like gravity, friction, momentum, collisions and projectile motion through direct observation. At the time the project will demonstrate how object-oriented programming works in practice. We will use class hierarchies, polymorphism, encapsulation, containers and spatial partitioning.

Month 1 – Core Engine and Object Model (Weeks 1–4)

- Design a PhysicsObject abstract base class.
- Create subclasses: Particle, Circle and Box.
- Build the simulation loop. Add gravity.
- Implement Vector2D operations for position, velocity and force calculations.
- Allow user input. Let users spawn objects into the scene.

Month 2 – Forces and Collision System (Weeks 5–8)

- Add forces like gravity, friction and spring forces using force generators.
- Develop broad-phase collision detection using a grid or quadtree to improve performance.
- Work on narrow-phase collision detection to find contact points between objects.
- Implement impulse-based collision response so objects react naturally when they collide.
- Add projectile motion. Include scenarios for testing.

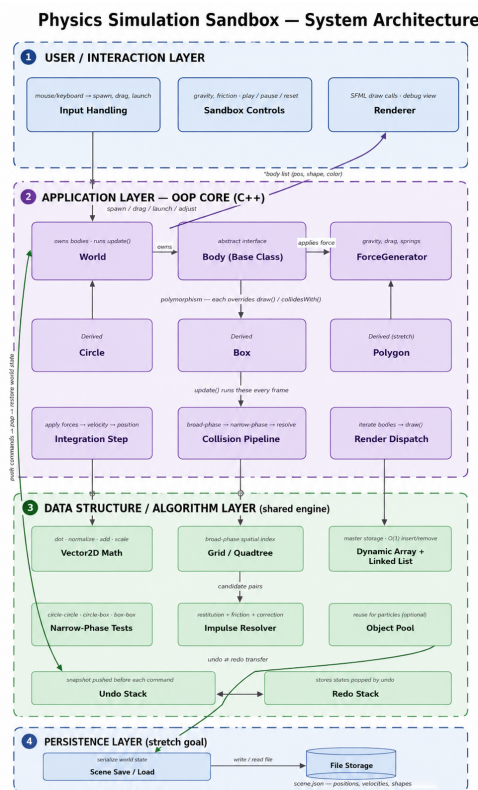
Month 3 – Interface, Testing and Finalization (Weeks 9–12)

- Finish the rendering system using SFML to clearly show all objects and interactions.
- Add. Load functionality so users can return to their setups later.
- Test the system thoroughly for both speed and accuracy in collision handling.
- Document the codebase. Prepare a demonstration to show everything working together.

Project Approach

The project will be built in C++17. Use SFML for drawing 2D graphics creating windows and handling user input. The system will follow an object-oriented design. We'll start with a `PhysicsObject` base class. Derived classes like `Particle`, `Circle` and `Box` will inherit from it showing inheritance, encapsulation, abstraction and polymorphism. Data structures will form the backbone of the simulation. Dynamic arrays and linked lists will manage physics objects. Stacks will keep track of states for undo. Queues will handle events during the simulation. A quadtree or spatial grid will divide the space to find objects near each other. Trees can organize objects by location to speed up searches. The physics engine will update these structures while calculating gravity, friction, projectile motion, momentum, collision detection and collision response. SFML will display the objects, their paths, vectors and live data. Users will be able to add, change, delete and interact with objects. They'll see how the data structures run the simulation behind the scenes. This approach keeps data structures at the center of object control, event handling, space searching and collision processing. OOP principles will run through every part of the system.

System Architecture



Project Outcome/Deliverables

The final result will be a working C++/SFML application that acts as a 2D physics sandbox. Students will be able to spawn particles, circles and boxes and watch gravity, friction, collisions, momentum and projectile motion unfold in time. The deliverables include:

- Complete C++ source code with the PhysicsObject class hierarchy.
- Simulation and numerical integration engine.
- Force generation system for gravity, friction and springs.
- Collision system with impulse-based response.
- Grid or quadtree spatial partitioning for collision processing.
- SFML-based user interface and rendering system.
- Preset demonstrations for /inelastic collisions, friction, gravity and projectile motion.
- Optional scene save/load feature.
- Final project demonstration.

Assumptions

- Users already know physics concepts like gravity, friction, momentum and collisions.
- A two-dimensional simulation is sufficient to show the core mechanics.
- Simplified physics is acceptable for learning purposes. Realism isn't required.
- The app will run on desktop computers with standard hardware.
- One user will use the sandbox at a time. No multiplayer or networking needed.
- The number of objects, in the simulation will usually stay under a hundred to maintain performance.

References

1. SFML (Simple and Fast Multimedia Library) – Official Documentation.
2. PhET Interactive Simulations, University of Colorado Boulder.
3. Algodoo – 2D Physics Sandbox.
4. Box2D – 2D Physics Engine.
5. Matter.js – 2D Physics Engine for JavaScript.
6. Ericson, C. *Real-Time Collision Detection*. CRC Press, 2004.
7. cppreference.com – C++ Standard Library Reference.
8. Millington, I. *Game Physics Engine Development*. CRC Press, 2010.

Web resources:

SFML: <https://www.sfm1-dev.org/documentation/>

PhET: <https://phet.colorado.edu/>

Algodoo: <https://www.algodoo.com/>

Box2D: <https://box2d.org/>

Matter.js: <https://brm.io/matter-js/>

cppreference: <https://en.cppreference.com/>